

Технологии программирования



Арсенал

Я!

Вопрос: Как обратиться к себе внутри класса?

```
this // указатель на себя
```

- Указатель на объект класса, с помощью которого вызвана функция.
- Можно использовать только внутри методов класса.

```
class MyClass  
{  
    void DoSomething(size_t number, std::string text);  
}
```

- В этом случае компилятор генерирует функцию с дополнительным параметром.

```
void DoSomething(MyClass* this, size_t number, std::string text); // thiscall
```

- Через this можно обращаться к любому методу или полю класса.

```
this->m_majorField = 10; // то же что и m_majorField = 10; в явном использовании this смысла нет
```

- Обычно используется как возвращаемое значение, например для цепочек a=b=c=d; чтобы цепочка была возможной, оператор = должен вернуть объект слева

```
MyClass& operator=(const MyClass& obj) { /*делаем приравнивание;*/ return *this;}
```

this недоступен из статических методов, так как они не требуют создания объекта класса для своего вызова.



Перезарядка не нужна

Методы по умолчанию - default

- Методы у которых существует стандартная реализация, компилятор может сам сгенерировать тело метода.

```
SomeClass& operator= (const SomeClass&) = default;
```

- Default - инструкция компилятору создать тело функции по умолчанию (стандартное)
- Позволяет определять стандартную реализацию для:
 - Конструкторов,
 - Деструкторов,
 - Базовых операторов.

```
MyClass(int value);           // наш конструктор с параметром. экранирует конструктор без параметров  
MyClass() = default;         // теперь конструктором без параметров можно пользоваться  
MyClass& operator=(const MyClass& obj) = default; // стандартная реализация оператора присваивания
```



Ничего лишнего



Удаленные методы - delete

- Запрет компилятору генерировать функции, у которых есть стандартная реализация
 - Конструкторы
 - Операторы
 - Деструкторы
- Явное указание компилятору на запрет генерацию и вызова функций, помеченных как удаленные

```
SomeClass& operator= (const SomeClass&) = delete;
```

- Механизм полезный для ограничения работы с классом, например запрет на копирование объекта класса. Упрощает работу с объектами класса и контроль памяти

```
MyClass& operator=(const MyClass&) = delete; // удалили оператор присваивания  
MyClass(const MyClass&) = delete;           // удалили конструктор копирования
```

- Позволяет удалить функцию класса родителя в классе наследнике.

```
struct MyClass {  
    void DoSomething();  
};  
  
struct MyClassChild : public MyClass {  
    void DoSomething() = delete;  
};
```

```
MyClassChild child;  
child.DoSomething(); // Error: функция явно удалена
```


Никакой автоматике



Явное указание - explicit

- Требование на явный вызов конструктора с именем класса. Запрет на использование листа инициализации.

```
explicit SomeClass(int a, bool isEnabled);  
...  
SomeClass errorObject = {1, true}; // Error. конструктор объявлен как явный, нужно указать тип  
SomeClass myObject    = SomeClass{1, true}; // вот так
```

- Указание имени класса требует только тот конструктор, который объявлен explicit.
- Если есть конструктор без модификатора explicit, то через него объект класса можно инициализировать обычным листом инициализации.
- Требование явного вызова static_cast для приведения типа. Запрет на неявное преобразование типа класса.

```
explicit operator char*()
```

- Теперь класс к типу char* можно привести только с явным использованием оператора приведения

```
MyClass my_class;  
char* errorString = my_class; // Error: требуется явное приведение типа  
char* string = static_cast<char*>(my_class); // вот так
```

Во имя скорости



Встраивание - inline

- Вызов функции занимает время.
- Для оптимизации компилятор может в месте вызова функции вставить ее исполняемый код.
- Функция должна быть короткой.
- Функция должна быть без внешних зависимостей, т.е. Она должна вызываться независимо.
- Можно проинструктировать компилятор, какую функцию считать встраиваемой.

```
class MyClass
{
    inline void DoSomething(); // функция будет встроена в код в месте вызова
};
```

- Встраиваемые функции нельзя отлаживать, встраивание происходит на уровне инструкций, т.е. после компиляции их код отсутствует.
- Постоянное использование inline раздувает финальный размер исполняемого файла.
- Можно заставить компилятор максимально оптимизировать вызовы, считая практически все функции inline - опция компилятора force-inline.

Внешний управление

Внешнее управление - volatile

- Компилятор старается сгенерировать наиболее оптимальный набор инструкций.
- Если код не может быть выполнен по определенным условиям, он будет выкинут.

```
volatile uint32_t variable = 0;
///  
variable = 10;  
// а что если эта переменная будет поменяна системой или в другом потоке? Тогда удалять if нельзя.  
if (variable < 5) // условие всегда истинно.  
{  
    // компилятор блок if оптимизирует. код для него сгенерирован не будет  
    variable = 0;  
}
```

- Volatile - инструкция компилятору, компилятор не будет оптимизировать код связанный с этой переменной.
- Волатильные переменные - переменные которые могут быть изменены в другом потоке или переданы системе, которая может их изменить извне программы.
- Компилятор не может предугадать какое значение переменная будет иметь в ходе выполнения программы.
- Волатильность - решение программиста.



Кое-что никогда не меняется

Константные методы - const

- Методы используются для получения данных о состоянии класса или расчетов не связанных с модификацией состояния класса.

```
int GetSomeData() const; // метод не может изменить значение полей класса
```

- Константным метод может быть только внутри класса.
- Внутри константного метода нельзя изменить значение полей класса, они константны.
- This так же константен. Компилятор сгенерирует следующую функцию:

```
int GetSomeData(const MyClass* this);
```

- Статический метод нельзя объявить константным.

Константные методы используются для

- Получения состояния класса - зачитывания его полей.
- Для выполнения операций, которые не меняют состояния класса.

Константный модификатор для методов помогает автору и пользователю кода понять, что вызов метода не изменит состояние класса.



Разрушитель устоев

Всегда модифицируемые поля - mutable

- Поле не может быть константой.

```
struct MyClass
{
    mutable int m_neverBeAConstantValue = 0; // поле никогда не будет константным
    int m_normalVariable = 5;                // обыкновенное поле структуры/класса
};
```

- Даже в составе константного объекта это поле может быть модифицировано.

```
const MyClass myClass;
myClass.m_neverBeAConstantValue = 10; // mutable поле можно менять всегда
myClass.m_normalVariable = 150;      // Егггг: обыкновенное поле у константного объекта тоже константа
```

- Это поле можно изменять внутри константных методов.



Менять состояние mutable полей внутри константных методов можно, но это плохой тон.

Не будите его!

Приватный конструктор

- Объект класса с приватным конструктором нельзя создать снаружи, используя стандартные подходы. `new/malloc/make_shared...`
- Позволяет контролировать процесс создания класса.
- Позволяет ограничить количество создаваемых объектов.
- Позволяет скрыть от пользователя сложную инициализацию, требующуюся для создания объекта класса.
- Позволяет создать класс наследник исходного класса в зависимости от внешних параметров, скрывая от пользователя процесс создания и какой конкретно наследник был создан.
- Объект такого класса может быть создан с помощью:
 - Объекта этого же класса
 - Статической функции этого класса (Функции фабрики)
 - Класа друга, (Класа Фабрики)

```
struct MyClass
{
private:
    MyClass() { ... };
```



Горнило

Шаблон проектирования: объект существующий в программе в одном экземпляре

```
class Singleton
{
public:

    static Singleton& Instance()
    {
        static Singleton sg;           // объект класса будет только один
        return sg;
    }

    Singleton(const Singleton&) = delete;    // Запрещаем копировать этот объект
    Singleton(const Singleton&&) = delete;   // Запрещаем перемещать объект
    Singleton operator=(const Singleton&) = delete; // Запрещаем присваивать этот объект
    ~Singleton() = default;
private:
    Singleton() {};                     // Запрещаем создавать объект снаружи класса
};
```

```
Singleton& object = Singleton::Instance(); // Обращение к синглтонному объекту
```



Синглтонов лучше избегать. Синглтон может считаться анти паттерном.

Миниатюры

Упаковка структуры или класса

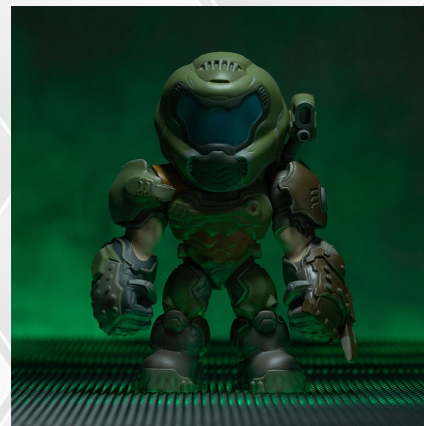
- Структура и класс занимают больше памяти чем сумма размеров их переменных.
- Короткие типа (<4 байт) выравниваются по 4 байта оставляя пустоты в структурах.
- 2х байтные и однобайтные выравниваются по четным байтам.

```
class PackedStructure {  
public:  
    char ch;        // 1 байт  
    // 0            дыра  
    // 0            дыра  
    // 0            дыра  
    int integer; // 4 байта  
};
```

```
class PackedStructure{  
public:  
    char ch;        // 1 байт  
    // 0            дыра  
    short shortValue; // 2 байта  
    int integer;     // 4 байта  
};
```

- pragma pack - инструкция компилятору упаковывать все структуры внутри pragma блока в соответствии с указанным выравниванием.
- Упаковывать можно несколько структур в рамках одного блока.

```
#pragma pack(push, 1) // Блок pragma, для упаковки полей структуры. Выравнивание - 1 байт  
class PackedStructure { // Структура/класс который будет упакован  
    // ...  
};  
#pragma pack(pop)     // конец блока упаковки структур
```



Фрагменты

Битовые поля

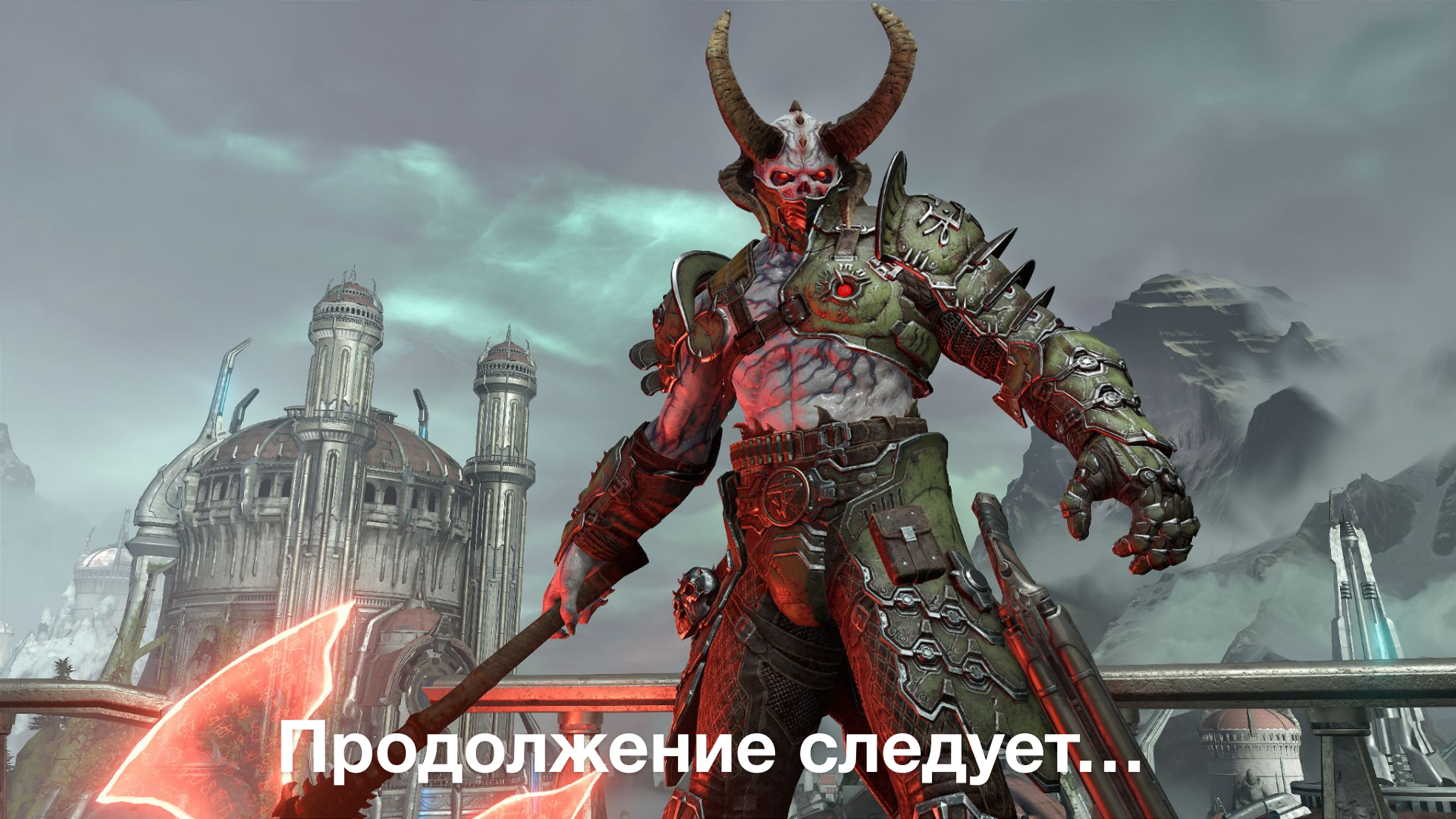
- Можно определить сколько бит будет занимать определенный тип.
- Битовые поля будут плотно упакованы, если суммарное количество занимаемых бит битовыми полями идущими подряд кратно 8ми битам.

```
int m_sign      : 1; // целое число в 1 бит (0 или 1)
int m_majorField : 4; // целое число в 4 бита
int m_minorField : 2; // целое число в 2 бита
int m_overflowBit : 1; // целое число в 1 бит
```

- Удобно, для упаковки и распаковки сложных нестандартных конструкций, со значащими битами.
- Позволяет избежать сложной битовой математики.

```
union rgb565 {
    short value;           // Числовое представление цвета
    struct {
        int r : 5;        // Компонент "красный" занимает первые 5 бит в числовом представлении
        int g : 6;        // Компонент "зеленый" занимает 6 бит с 5-го по 11 в числовом представлении
        int b : 5;        // Компонент "синий" занимает последние 5 бит в числовом представлении
    } bitColor;
};
```





Продолжение следует...